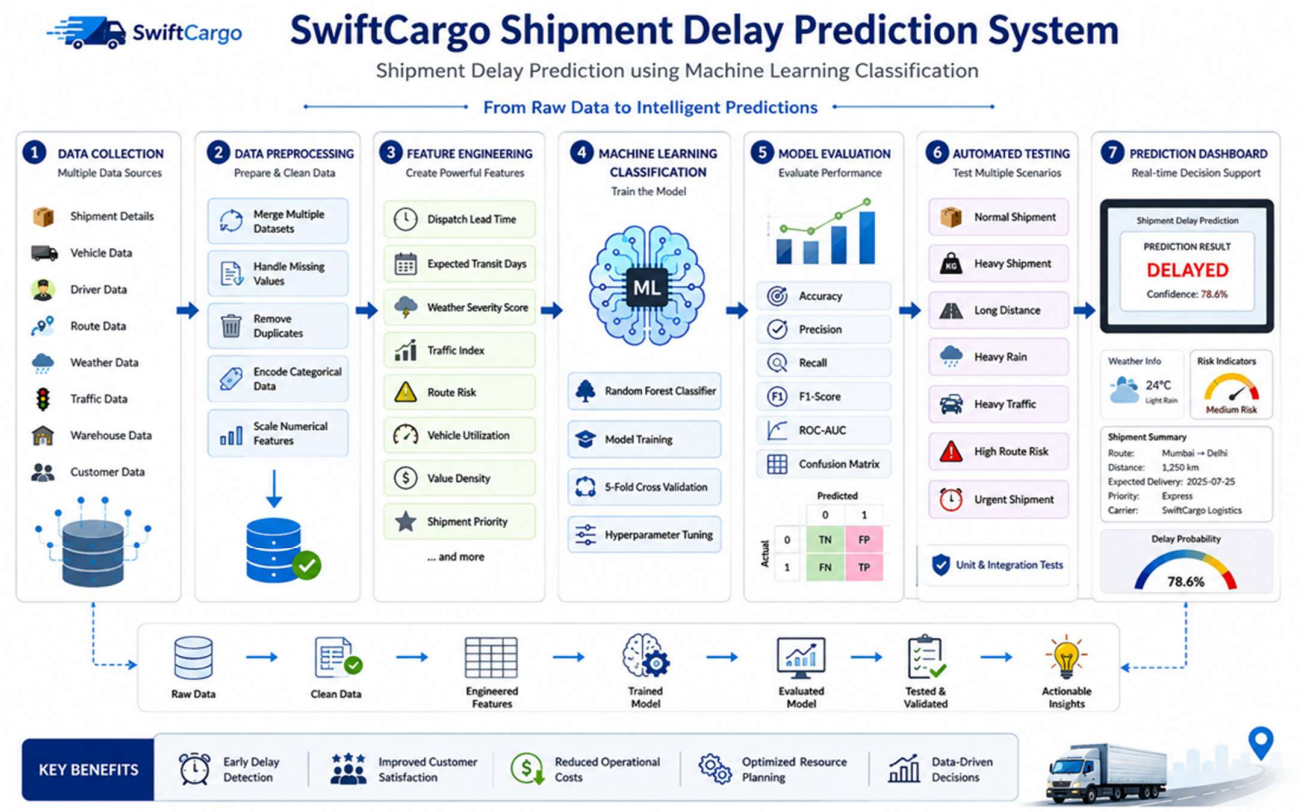
SwiftCargo Shipment Delay Prediction System

An Intelligent Machine Learning-Based Solution for Predicting Shipment Delays Using Feature Engineering, Model Evaluation, and Automated Testing

Prepared For: SwiftCargo Shipment Delay Prediction System – Data Science Team, Logistics Industry Professionals, Supply Chain Analysts, Project Mentors, and Academic Evaluators

Prepared By: Patil Devendra Vinod

Date: July 2026



Executive Summary

The Shipment Delay Prediction System using Machine Learning is an intelligent logistics analytics solution developed to predict whether a shipment is likely to be delayed before its scheduled delivery. The project aims to help logistics companies identify high-risk shipments in advance, enabling proactive decision-making, reducing operational costs, and improving customer satisfaction. An integrated logistics dataset containing shipment, vehicle, route, warehouse, carrier, weather, traffic, and customer information was used for model development. Data preprocessing techniques such as handling missing values, removing duplicates, treating outliers, and encoding categorical variables were applied to ensure high-quality data. Exploratory Data Analysis (EDA) identified key factors influencing shipment delays, including traffic congestion, weather conditions, transportation distance, dispatch lead time, warehouse utilization, and carrier performance.

To improve prediction accuracy, feature engineering was performed by creating meaningful variables such as expected transit days, traffic severity, weather indicators, and time-based features. Multiple machine learning algorithms, including Decision Tree, Random Forest, Logistic Regression, K-Nearest Neighbors (KNN), Naïve Bayes, Support Vector Machine (SVM), and XGBoost, were trained and evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. After optimization using GridSearchCV and RFECV, the Random Forest Classifier achieved the best performance with an Accuracy of 71.28%, Precision of 73.68%, and a ROC-AUC score of 72.46%. The final model was deployed as an interactive Streamlit web application that provides real-time shipment delay predictions and supports data-driven logistics decision-making.

Introduction

The logistics and supply chain industry plays a vital role in connecting manufacturers, distributors, and customers by ensuring the timely movement of goods. With the rapid growth of e-commerce and global trade, logistics companies manage thousands of shipments every day across different routes, warehouses, and transportation networks. Delivering shipments on or before the expected delivery date is essential for maintaining customer satisfaction, reducing operational costs, and improving overall business performance. However, shipment delays remain one of the most significant challenges faced by logistics organizations. Factors such as traffic congestion, adverse weather conditions, inefficient route planning, vehicle breakdowns, warehouse congestion, and operational constraints can disrupt delivery schedules and negatively impact supply chain efficiency. These challenges highlight the need for intelligent systems that can predict potential delays before they occur.

Machine Learning provides a powerful approach for solving this problem by analyzing historical logistics data and identifying patterns associated with delayed deliveries. Unlike traditional rule-based methods, machine learning models can process large volumes of operational, environmental, and transportation data to estimate the probability of shipment delays with greater accuracy. The SwiftCargo Shipment Delay Prediction System was developed to leverage these capabilities by integrating shipment, vehicle, route, warehouse, carrier, traffic, weather, and customer information into a unified predictive analytics framework. The project includes data preprocessing, exploratory data analysis (EDA), feature engineering, feature selection, model development, hyperparameter tuning, and performance evaluation to build a reliable delay prediction model.

To identify the most suitable prediction model, several supervised machine learning algorithms, including Decision Tree, Random Forest, Logistic Regression, K-Nearest Neighbors (KNN), Naïve Bayes, Support Vector Machine (SVM), and XGBoost, were implemented and compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. After optimization through GridSearchCV and Recursive Feature Elimination with Cross-Validation (RFECV), the Random Forest Classifier was selected as the final model due to its superior performance. The trained model was deployed as an interactive Streamlit web application that enables users to perform real-time shipment delay predictions. By identifying high-risk shipments in advance, the proposed system supports proactive decision-making, improves resource allocation, reduces transportation costs, minimizes supply chain disruptions, and enhances customer satisfaction, demonstrating the practical application of artificial intelligence in modern logistics.

Problem Statement

The logistics and supply chain industry depends on timely shipment delivery to maintain operational efficiency and customer satisfaction. However, shipments are frequently delayed due to factors such as traffic congestion, adverse weather conditions, vehicle breakdowns, warehouse congestion, inefficient route planning, and documentation issues. These unpredictable factors make it difficult for logistics companies to accurately identify shipments that are at risk of delay. As shipment volumes continue to increase, relying on manual monitoring and traditional decision-making methods becomes inefficient, time-consuming, and prone to errors.

Shipment delays have significant operational and financial consequences for logistics organizations. Delayed deliveries increase transportation and warehouse costs, reduce fleet and resource utilization, generate customer complaints, and may result in Service Level Agreement (SLA) violations. They also disrupt supply chain operations, damage the company's reputation, and reduce customer trust. Without an effective prediction mechanism, logistics managers can only respond after delays occur, limiting their ability to take preventive actions and optimize delivery operations.

To address these challenges, this project proposes the SwiftCargo Shipment Delay Prediction System, an intelligent machine learning-based solution that predicts whether a shipment is likely to be delayed before delivery. By analyzing historical shipment records along with vehicle, route, warehouse, traffic, weather, and operational data, the system identifies high-risk shipments and provides early predictions. This enables logistics companies to optimize route planning, allocate resources more efficiently, reduce transportation costs, minimize delivery delays, and improve overall customer satisfaction through proactive, data-driven decision-making.

Project Scope

The scope of this project is to develop a machine learning-based system that predicts shipment delays using historical logistics data. It includes data preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and optimization. The system analyzes shipment, vehicle, route, warehouse, weather, traffic, and carrier information to generate accurate predictions. The final model is deployed as an interactive Streamlit web application for real-time shipment delay prediction. The project supports proactive logistics planning, efficient resource allocation, and improved decision-making in supply chain operations.

Project Objectives

The primary objective of the SwiftCargo Shipment Delay Prediction System is to develop an intelligent machine learning model capable of predicting whether a shipment is likely to be delayed before its scheduled delivery. The project aims to analyze historical shipment data along with route, vehicle, warehouse, weather, traffic, carrier, and customer information to identify high-risk shipments. By providing early delay predictions, the system enables logistics companies to take proactive actions, improve delivery planning, optimize resource allocation, and reduce transportation costs. The project also focuses on improving data quality through preprocessing, exploratory data analysis (EDA), feature engineering, and feature selection to build an accurate and reliable predictive model.

Another important objective is to compare multiple supervised machine learning algorithms, including Decision Tree, Random Forest, Logistic Regression, K-Nearest Neighbors (KNN), Naïve Bayes, Support Vector Machine (SVM), and XGBoost, and identify the best-performing model through hyperparameter tuning and RFECV feature selection. The selected model is deployed as an interactive Streamlit web application that provides real-time shipment delay predictions with confidence scores. The overall goal is to support data-driven decision-making, enhance operational efficiency, minimize supply chain disruptions, improve customer satisfaction, and demonstrate the practical application of machine learning in modern logistics and supply chain management

System Architecture

The Shipment Delay Prediction System is designed using a modular architecture that integrates data collection, preprocessing, machine learning, and result visualization into a unified web application. The system is developed using Streamlit as the front-end interface, where users can enter shipment information or upload CSV files for batch prediction. The application also connects to the OpenWeatherMap API to automatically retrieve real-time weather information based on the selected destination city. This architecture combines shipment, route, vehicle, warehouse, carrier, operational, and environmental data to provide a comprehensive input for delay prediction.

The application layer performs input validation and executes the preprocessing pipeline before prediction. All user inputs are validated for completeness, acceptable ranges, and data consistency. The validated data then undergoes preprocessing, including missing value imputation using dataset median values, categorical encoding, feature engineering, one-hot encoding, and alignment with the 30 selected features used during model training. The processed feature vector is supplied to the optimized Random Forest Classifier, which generates both the shipment status and the probability of delay. The same preprocessing pipeline is applied consistently for both single-shipment and batch CSV predictions to ensure reliable and standardized results.

The presentation layer displays prediction results through an interactive dashboard that provides shipment status, delay probability, confidence level, shipment summary, risk interpretation, and operational recommendations. For batch predictions, the system additionally generates summary statistics such as the total number of shipments, delayed and on-time shipments, overall delay percentage, and average delay probability. Users can also download prediction results as CSV files for reporting and further analysis. This layered architecture ensures scalability, consistency, and efficient decision support while enabling logistics managers to make proactive operational decisions based on real-time and data-driven insights.

Data Flow Architecture

The data flow architecture begins with the user entering shipment details through the Streamlit web interface or uploading a CSV file for batch prediction. The system retrieves real-time weather information from the OpenWeatherMap API and combines it with the user-provided shipment data. The integrated data is validated to ensure completeness, consistency, and correctness before processing. Next, the preprocessing pipeline performs missing value handling, feature engineering, categorical encoding, and feature alignment with the trained model. The processed feature vector is then passed to the optimized Random Forest classifier to generate shipment delay predictions and probability scores. Finally, the prediction results, analytics, and recommendations are displayed on the dashboard, with an option to download the results as a CSV report.

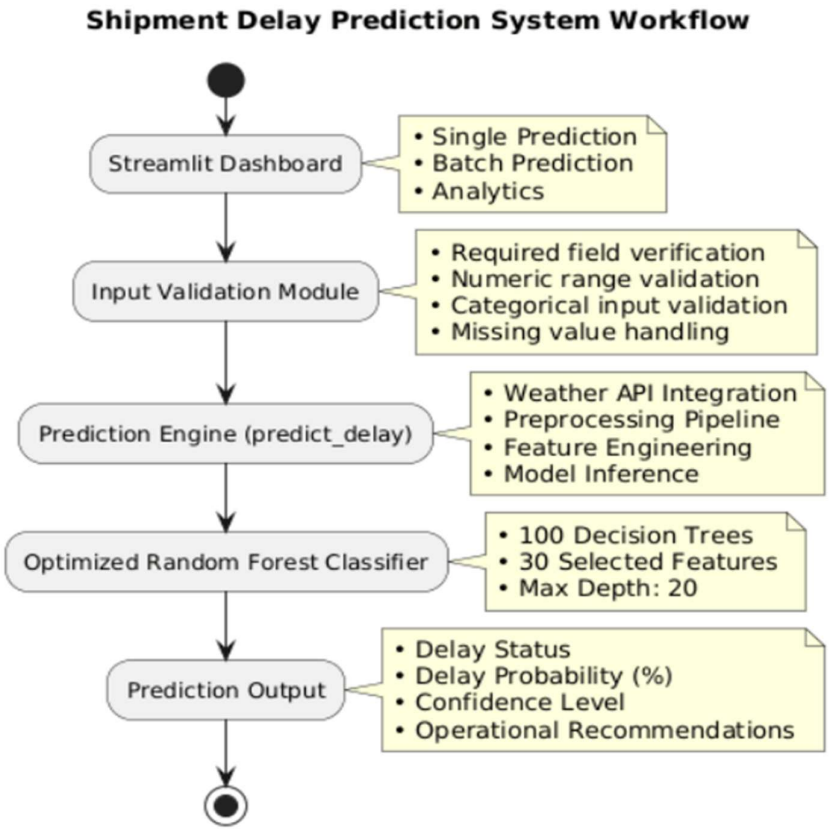


Figure 2: Component Overview

Component Overview

The system consists of the following key components:

- **Streamlit Dashboard** – User interface for entering shipment details and viewing predictions.
- **Input Validation Module** – Validates user inputs before processing.
- **Weather API Integration** – Retrieves real-time weather information.
- **Data Preprocessing Module** – Cleans and prepares the input data.
- **Feature Engineering Module** – Generates derived features for improved prediction.
- **Prediction Engine (predict_delay)** – Processes the input and performs model inference.
- **Optimized Random Forest Classifier** – Predicts shipment delay using the trained machine learning model.
- **Prediction Output Dashboard** – Displays delay status, probability, confidence level, and recommendations.

Technology Stack

Programming Language: The application was developed using Python 3.x, which provides a robust ecosystem for machine learning, data analysis, and web application development.

Web Framework: Streamlit was used to build the interactive web interface, enabling users to perform shipment delay predictions through a simple and user-friendly dashboard.

Machine Learning: The predictive model was developed using the Scikit-learn library, which provides efficient implementations of machine learning algorithms and model evaluation techniques.

Data Processing: Pandas and NumPy were used for data cleaning, preprocessing, feature engineering, and numerical computations throughout the project.

Visualization: Matplotlib and Plotly were utilized to generate charts and interactive visualizations for data analysis and prediction insights.

Weather Data: Real-time weather information was retrieved using the OpenWeatherMap API, allowing weather conditions to be incorporated into shipment delay predictions.

Model Serialization: The trained machine learning model and preprocessing objects were saved using Joblib/Pickle, enabling efficient model loading and deployment.

Version Control: Git was used for version control to manage source code, track changes, and support collaborative development throughout the project.

Dataset Overview

The SwiftCargo Shipment Delay Prediction System was developed using an integrated logistics dataset that represents the complete shipment lifecycle, from order placement to final delivery. The dataset was created by integrating multiple operational data sources that simulate real-world logistics activities, including shipment management, transportation, warehouse operations, weather conditions, traffic information, and customer requirements. This integrated approach enables the machine learning model to capture the combined influence of operational, environmental, and transportation-related factors on delivery performance. By bringing together information from multiple business domains, the system can identify complex relationships associated with shipment delays and improve the accuracy of delay prediction.

Data Sources

The dataset consists of multiple structured CSV files that represent different aspects of logistics operations.

Data Source	Description
Shipment Details	Core shipment information
Time Details	Temporal attributes
Location Details	Origin and destination information
Transport Details	Vehicle and transportation data
Carrier Details	Carrier performance metrics
Package Details	Package characteristics
Weather Details	Environmental conditions
Traffic Details	Road congestion information
Warehouse Details	Storage and handling data
Operations Details	Operational metrics
Customer Preferences	Customer requirements
Target Delivery Outcome	Delay status (target variable)

Dataset Structure

Dataset Information	Description
Dataset Type	Integrated Logistics Dataset
Source Files	Multiple CSV files
Primary Key	Shipment_ID
Prediction Target	Delay_Flag

Feature Description

The integrated dataset contains both numerical and categorical features that describe shipment characteristics, transportation conditions, warehouse operations, customer information, weather conditions, and traffic status. These features provide valuable information for understanding the factors influencing shipment delays and enable the machine learning models to make accurate predictions.

Numerical Features

The numerical features represent measurable operational and environmental characteristics used during model training. These include:

- Shipment Weight
- Shipment Volume
- Declared Value
- Distance Covered
- Vehicle Capacity
- Vehicle Age
- Fleet Size
- Driver Experience
- Warehouse Capacity
- Warehouse Utilization
- Average Transit Days

- Traffic Index
- Temperature
- Rainfall
- Humidity
- Wind Speed
- Visibility
- Dispatch Lead Time
- Expected Transit Days
- Delivery Efficiency Indicators

Categorical Features

Categorical features describe qualitative operational conditions that require encoding before being used by machine learning algorithms. The major categorical attributes include:

- Shipment Priority (Express, Standard, Economy)
- Shipment Type (Domestic, Import, Export)
- Weather Condition (Clear, Rain, Fog, Storm)
- Route Risk Level (Low, Medium, High)
- Maintenance Status (Good, Needs Maintenance)
- Documentation Status (Complete, Incomplete)
- Vehicle Utilization Category

Target Variable

The objective of this project is to predict whether a shipment will be delivered on time or experience a delay. Therefore, the target variable selected for model development is Delay_Flag, which represents a binary classification problem.

Value	Meaning
0	On-Time Shipment
1	Delayed Shipment

Data Quality Assessment

Before developing the prediction models, the integrated logistics dataset underwent a comprehensive data quality assessment to ensure that it was suitable for machine learning. The assessment focused on identifying inconsistencies, missing information, and data-related issues that could affect model performance. The following quality checks were performed:

- Identification of missing values in operational, transportation, and environmental attributes.
- Detection and removal of duplicate shipment records generated during dataset integration.
- Identification of extreme values (outliers) in numerical variables such as shipment weight, declared value, rainfall, and transportation distance.
- Verification and correction of inconsistent data types, particularly date and time attributes.
- Identification of categorical variables requiring numerical encoding.
- Analysis of feature scale variations among numerical attributes.

These quality checks ensured that the dataset was reliable, consistent, and suitable for predictive modeling.

Data Preprocessing Pipeline

A comprehensive data preprocessing pipeline was implemented to transform the raw logistics dataset into a clean, structured, and machine-learning-ready dataset. The preprocessing process improved data quality, increased consistency, reduced noise, and enhanced the predictive performance of the machine learning models.

Missing Value Handling

Missing values were analyzed before model development to determine the most appropriate imputation strategy. For numerical attributes, the Iterative Imputer (MICE) was applied during model training because it estimates missing values using relationships among multiple variables. During deployment, however, median values were used since iterative imputation is not suitable for predicting individual records. Missing values in categorical variables were replaced using the most frequent category (mode). This approach preserved valuable information while minimizing the impact of incomplete records.

Duplicate Record Removal

Duplicate shipment records created during dataset integration were identified and removed to eliminate redundant observations. Removing duplicate records prevented bias in the training dataset and ensured that each shipment contributed only once during model learning.

Outlier Treatment

Extreme values were identified using the Interquartile Range (IQR) method. The process involved calculating the first quartile (Q1), third quartile (Q3), and the Interquartile Range (IQR). Lower and upper bounds were then determined using the standard IQR rule, and observations falling outside these limits were considered outliers. This analysis helped identify unusual shipment behavior while reducing the influence of abnormal observations on model performance.

Categorical Encoding

Since machine learning algorithms require numerical inputs, categorical variables were transformed into numerical representations. One-Hot Encoding was applied to nominal variables such as Shipment Type and Weather Condition, while Label Encoding was used for ordinal variables such as Shipment Priority and Route Risk Level.

Date Transformation

Date-related attributes were converted into a standard datetime format before analysis. Additional temporal features were then generated from these date fields, including Dispatch Lead Time, calculated as the difference between Dispatch Date and Order Date, and Expected Transit Days, calculated as the difference between Expected Delivery Date and Dispatch Date. These engineered time-based features provided useful information about shipment scheduling and operational efficiency.

Feature Engineering

Feature engineering was performed to improve the predictive capability of the machine learning models by creating meaningful variables from existing attributes. Important engineered features included:

- Vehicle Utilization, calculated as Shipment Weight divided by Vehicle Capacity, representing how efficiently vehicle capacity is utilized.
- Value Density, calculated as Declared Value divided by Shipment Weight, indicating the monetary value of goods per kilogram.
- Weight per Unit, calculated as Shipment Weight divided by the Number of Units, representing the average weight of each package.
- Dispatch Lead Time, calculated as the time difference between Order Date and Dispatch Date, indicating the time taken to dispatch a shipment after booking.

These engineered features enabled the models to capture hidden operational relationships that were not directly available in the original dataset.

Class Imbalance Handling

The distribution of delayed and on-time shipments was examined before model training to identify any imbalance between the target classes. Appropriate balancing techniques were applied to reduce bias toward the majority class and improve the model's ability to accurately identify delayed shipments.

Feature Scaling

Numerical features were standardized using StandardScaler, particularly for algorithms that are sensitive to feature magnitude, such as Logistic Regression. Standardization ensured that all numerical variables were measured on a common scale, improving model convergence and overall predictive performance.

Summary

Data preprocessing was one of the most critical stages in developing the SwiftCargo Shipment Delay Prediction System. Through systematic handling of missing values, duplicate record removal, outlier treatment, categorical encoding, date transformation, feature engineering, class imbalance handling, and feature scaling, the raw logistics data was transformed into a clean, structured, and machine-learning-ready dataset. This preprocessing pipeline significantly improved data quality and provided a reliable foundation for training accurate, robust, and efficient machine learning models.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted to understand the characteristics of the integrated logistics dataset and identify the major factors influencing shipment delays. Statistical analysis and visualizations were used to

examine the distribution of variables, relationships among features, and operational patterns before model development.

The analysis revealed that transportation distance, traffic congestion, weather conditions, route risk, and dispatch lead time were among the most influential factors affecting delivery performance. Longer transportation distances and adverse weather conditions were associated with higher delay probabilities, while high traffic congestion and risky routes significantly increased travel uncertainty. These findings highlighted the importance of considering multiple operational and environmental factors when predicting shipment delays.

Additionally, EDA confirmed that data preprocessing and feature engineering improved the quality and representation of the dataset. Newly engineered features such as Dispatch Lead Time, Vehicle Utilization, Value Density, and Weight per Unit captured important logistics characteristics and enhanced the predictive capability of the machine learning models. Overall, the insights obtained from EDA provided a strong foundation for feature selection, model training, and accurate shipment delay prediction.

Machine Learning Approach

Model Development Methodology

The objective of this project was to develop a machine learning model capable of predicting shipment delays before delivery. A three-phase methodology was adopted: (1) initial model comparison, (2) hyperparameter tuning, and (3) feature selection. All models were trained using an 80:20 stratified train-test split and evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC to ensure a fair comparison.

Initial Model Comparison

Seven classification algorithms were evaluated to identify the most suitable model for shipment delay prediction.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	60.20%	45.45%	43.82%	44.62%	56.74%
Random Forest	69.10%	70.97%	26.29%	38.37%	70.82%
Logistic Regression	66.33%	61.90%	20.72%	31.04%	66.75%
KNN	52.77%	28.40%	19.12%	22.86%	43.27%
Naive Bayes	65.01%	57.14%	17.53%	26.83%	60.81%
XGBoost	68.37%	59.14%	43.82%	50.34%	66.28%
SVM	69.10%	65.85%	32.27%	43.32%	70.74%

Random Forest and SVM achieved the highest accuracy (69.10%), while XGBoost obtained the highest Recall and F1-Score. Based on the overall performance, Random Forest, XGBoost, and Logistic Regression were selected for further optimization. SVM was excluded due to its higher computational cost and lower interpretabilit

Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with 5-Fold Cross-Validation to identify the optimal parameter combination for each selected model. Different combinations of parameters were evaluated, and the configuration with the highest cross-validation score was selected.

Best Hyperparameters

Model	Best Parameters
Random Forest	max_depth=20, min_samples_leaf=1, min_samples_split=2, n_estimators=100
XGBoost	learning_rate=0.1, max_depth=3, n_estimators=200, subsample=1.0
Logistic Regression	C=0.1, penalty=L2, solver=liblinear

Performance After Tuning

Hyperparameter tuning improved the performance of the selected models. Random Forest achieved the highest accuracy (70.41%) and precision (74.49%), while Logistic Regression obtained the highest recall (38.65%) and F1-score (47.78%). XGBoost showed a slight decrease in accuracy but remained competitive. Overall, Random Forest remained the best-performing model after tuning.

Feature Selection Using RFECV

Recursive Feature Elimination with Cross-Validation (RFECV) was applied to remove less important features and identify the optimal feature subset. Using a Random Forest estimator with Stratified 5-Fold Cross-Validation, RFECV selected 30 important features, reducing model complexity while improving prediction performance.

Performance After RFECV

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	71.28%	73.68%	33.47%	46.03%	72.46%
Logistic Regression	69.39%	64.34%	36.65%	46.70%	70.16%
XGBoost	68.66%	60.98%	39.84%	48.19%	—

RFECV improved the performance of all selected models by eliminating irrelevant features. Random Forest achieved the highest Accuracy and ROC-AUC, while XGBoost obtained the highest Recall and F1-Score.

Performance Evolution

The performance of the selected models improved through hyperparameter tuning and RFECV feature selection. Random Forest showed consistent improvement, with accuracy increasing from 69.10% to 70.41% after tuning and 71.28% after RFECV (+2.18%). Logistic Regression improved from 66.33% to 69.39% (+3.06%), while XGBoost showed a slight overall improvement from 68.37% to 68.66% (+0.29%). Overall, Random Forest achieved the highest final accuracy and demonstrated the most consistent performance across all stages.

Final Model Selection

After evaluating multiple machine learning algorithms and applying hyperparameter tuning and RFECV-based feature selection, the Random Forest Classifier was selected as the final model. It achieved the best overall performance with an Accuracy of 71.28%, Precision of 73.68%, and ROC-AUC of 72.46%, outperforming the other models. Its ability to handle complex relationships, reduce overfitting through ensemble learning, and provide reliable predictions makes it well suited for shipment delay prediction. Therefore, the Random Forest model was deployed as the final prediction model in the SwiftCargo Shipment Delay Prediction System.

System Implementation

System Overview

The SwiftCargo Shipment Delay Prediction System was implemented as an interactive web application using Streamlit to provide real-time shipment delay prediction. The application integrates data preprocessing, feature engineering, machine learning, and weather information into a single platform. Users can enter shipment details through a user-friendly interface or upload a CSV file for batch prediction. The system processes the input data using the same preprocessing pipeline applied during model training and generates shipment delay predictions using the optimized Random Forest Classifier. This implementation enables logistics managers to identify high-risk shipments before dispatch and supports data-driven operational decision-making.

System Workflow

The workflow begins when the user enters shipment information or uploads a CSV file through the Streamlit dashboard. The application validates the input data and automatically retrieves real-time weather information using the OpenWeatherMap API based on the destination city. The collected data then undergoes preprocessing, including missing value handling, categorical encoding, feature engineering, and feature alignment with the selected model features. The processed data is passed to the trained Random Forest model, which predicts whether the shipment is likely to be delayed and calculates the corresponding delay probability. Finally, the prediction results, confidence level, and operational recommendations are displayed on the dashboard.

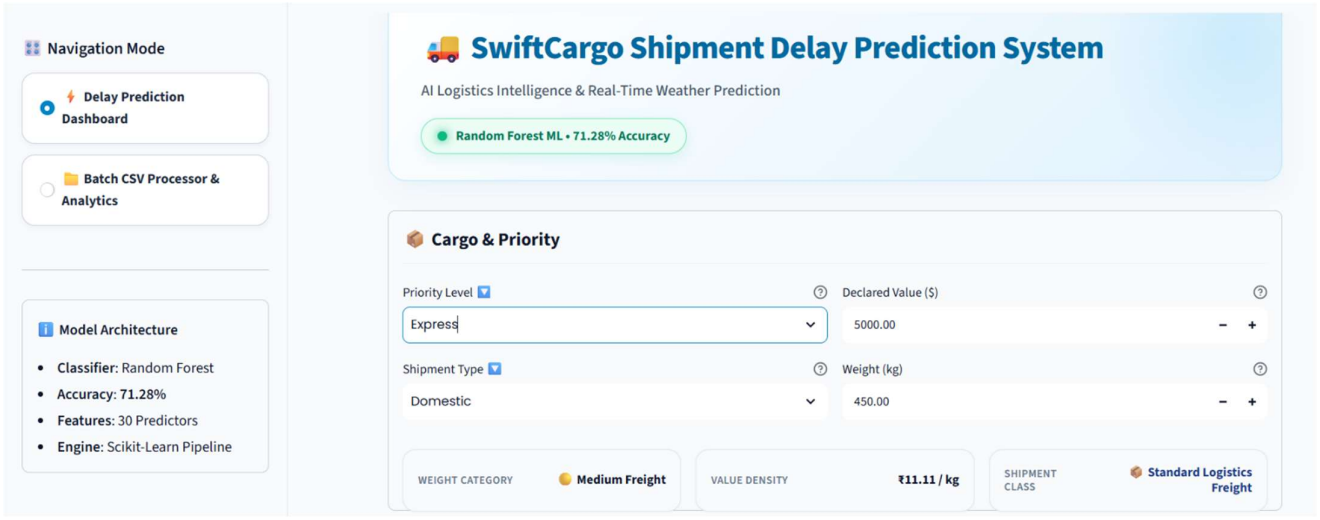
Application Features

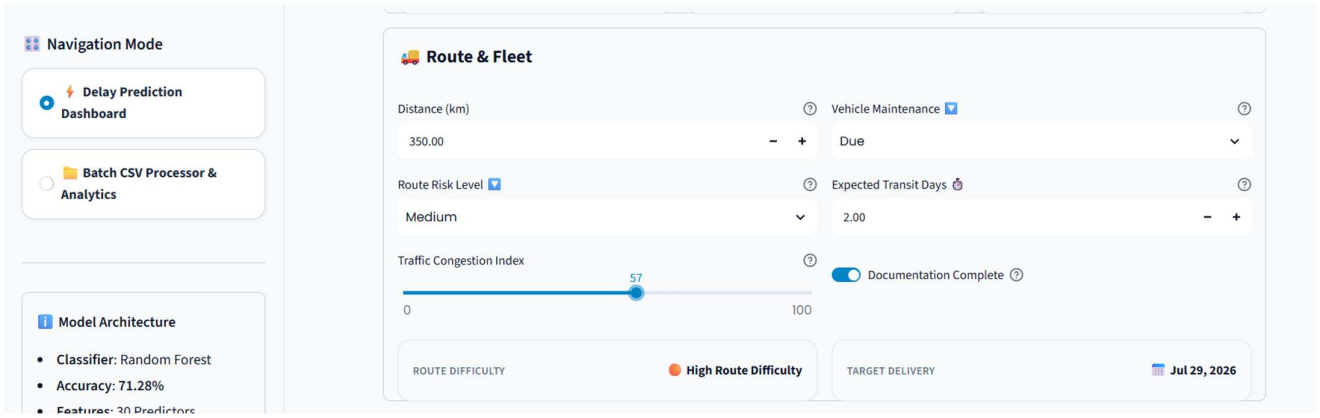
The application provides several features to support shipment delay prediction and logistics planning. It supports both single shipment prediction and batch prediction through CSV file uploads. The system automatically integrates weather information into the prediction process and displays the shipment status, delay probability, confidence level, and operational recommendations. Users can also download the prediction results as a CSV file for reporting and further analysis. These features make the application practical and suitable for real-world logistics operations.

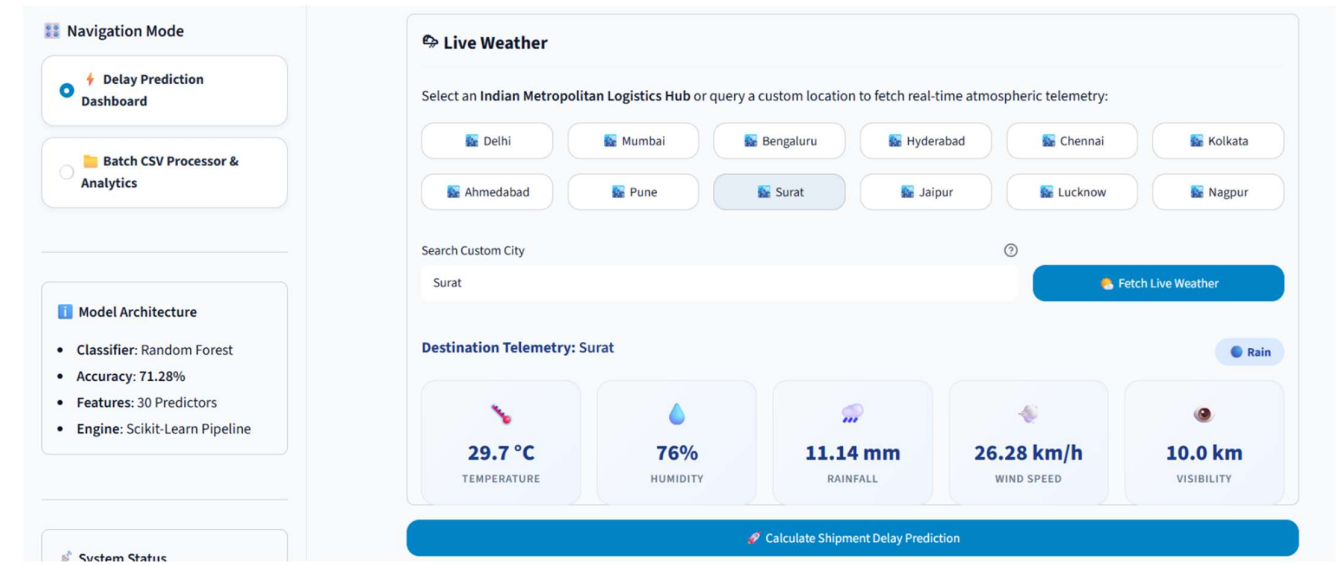
Deployment

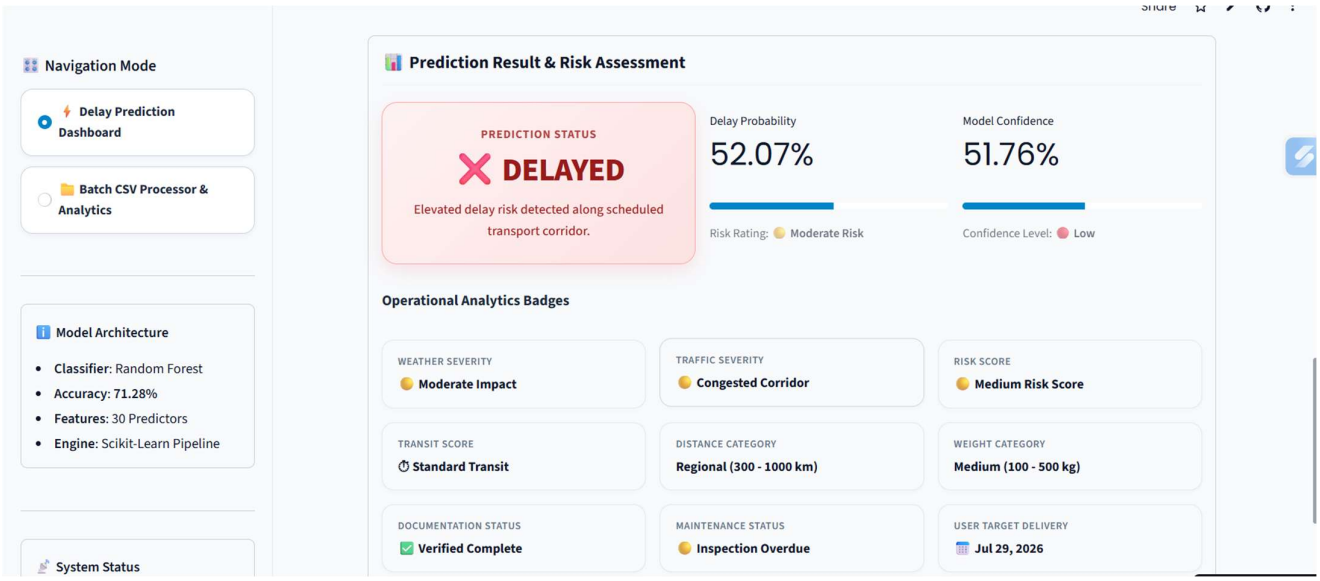
The trained Random Forest model and preprocessing objects were serialized using Joblib and integrated into the Streamlit application. The deployed system uses the same preprocessing steps and selected features as the training pipeline, ensuring consistent and reliable predictions for both individual shipments and batch processing. This deployment demonstrates the practical application of machine learning for solving real-world shipment delay prediction problems

Final Application Interface
Delay Prediction Dashboard

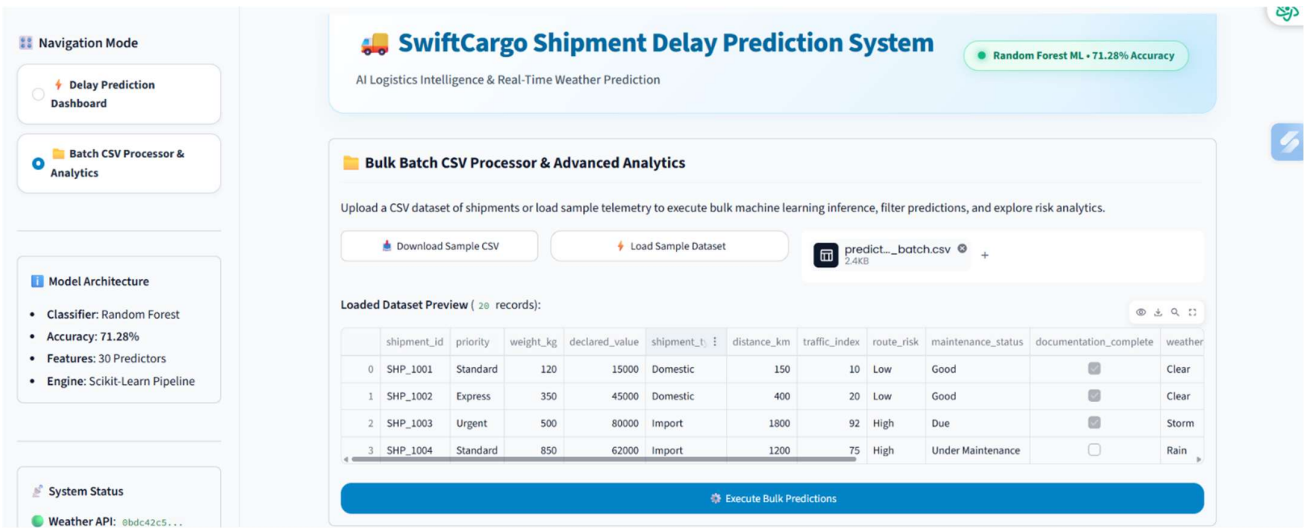


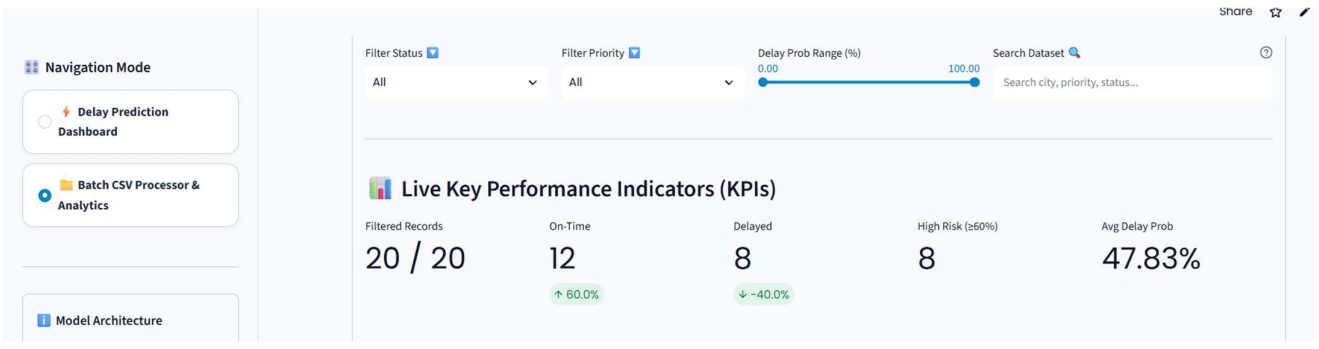


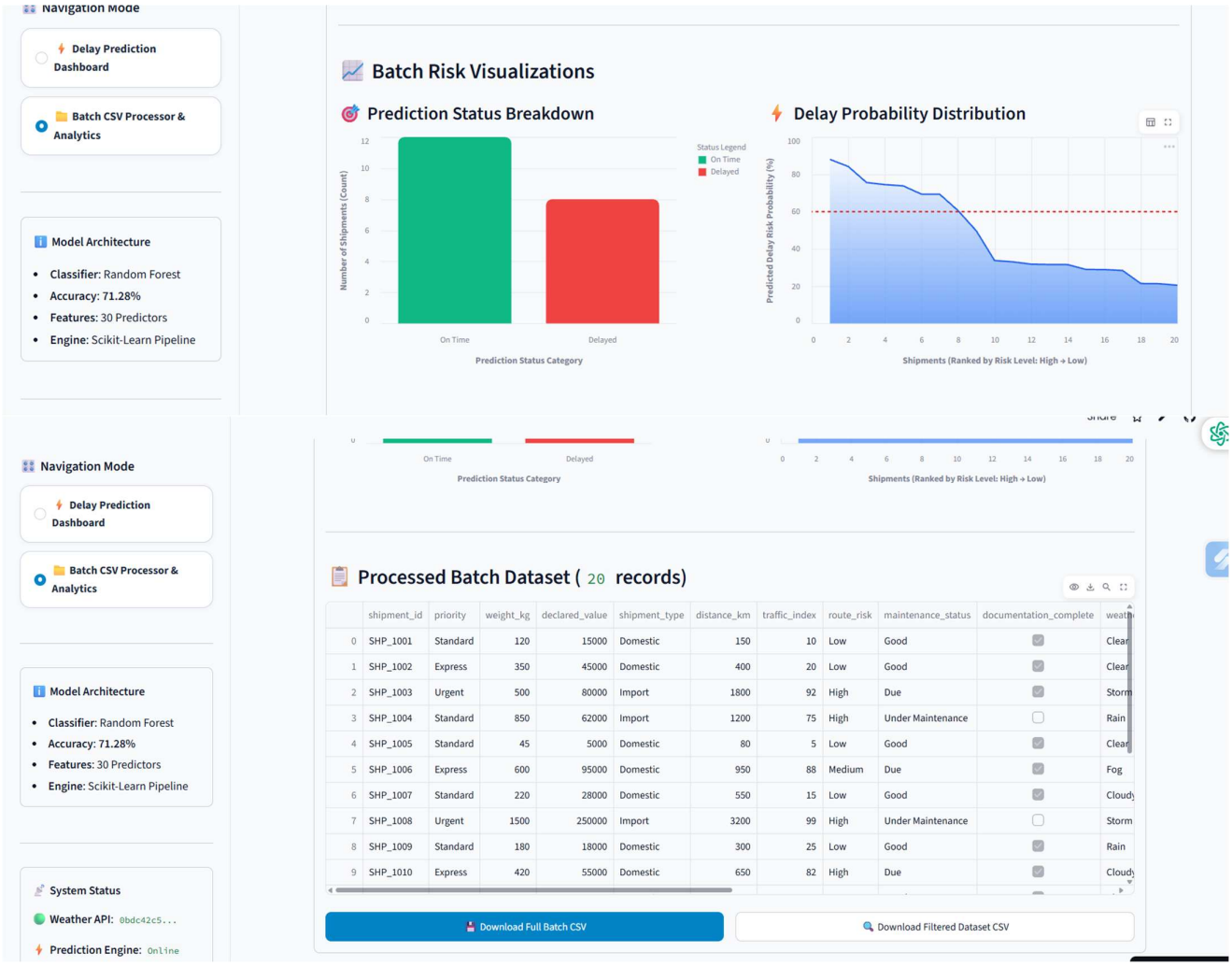




Batch Csv Processor and Analytics







Challenges Faced and Solutions Implemented

During the development of the **SwiftCargo Shipment Delay Prediction System**, several technical and operational challenges were encountered while building, training, and deploying the machine learning model. These challenges were systematically addressed through appropriate preprocessing techniques, robust software design, and validation procedures to ensure that the application remained reliable, accurate, and suitable for real-world logistics operations.

Missing Values in Live Prediction Inputs

One of the major challenges was handling missing values during real-time prediction. Users occasionally submitted incomplete shipment information through the Streamlit dashboard or batch CSV files, which could lead to prediction failures. To address this issue, a fallback imputation mechanism based on dataset median values was implemented. Median values calculated from the training dataset were stored in a predefined dictionary and automatically used whenever a numerical feature was missing. This approach prevented prediction failures, improved the robustness of the prediction pipeline, and ensured reliable predictions even when partial input information was provided.

Training–Prediction Feature Consistency

Another challenge involved maintaining consistency between the training dataset and live prediction inputs. The Random Forest model was trained using 30 selected features arranged in a specific order, whereas user inputs from the dashboard differed from the training data structure. To overcome this issue, a centralized preprocessing function was developed to perform feature engineering, categorical encoding, missing value handling, and feature alignment before prediction. The function automatically reordered all features to match the structure expected by the trained model, thereby eliminating feature mismatch errors and ensuring consistent predictions during deployment.

Inconsistent Categorical Inputs

Users often entered categorical values such as shipment priority and maintenance status in different formats, including lowercase, uppercase, or mixed-case text. These inconsistencies caused encoding errors during preprocessing. To solve this problem, robust mapping dictionaries were implemented to standardize multiple input formats into consistent numerical representations. This standardization eliminated encoding-related errors, improved preprocessing accuracy, and increased the application's compatibility with different user input styles.

Weather API Failures

The application integrates real-time weather information using the OpenWeatherMap API, making it dependent on internet connectivity and external services. Temporary network failures, DNS issues, or API timeouts occasionally interrupted weather retrieval. To maintain uninterrupted operation, exception handling was implemented around all API requests, and realistic default weather values were supplied whenever the API was unavailable. Additionally, Streamlit caching was used to reduce unnecessary API requests and improve application performance. These measures prevented application crashes and ensured stable prediction even under poor network conditions.

Feature Engineering During Live Prediction

Several important predictive features, including Vehicle Utilization and Value Density, were not directly entered by users but were required by the trained machine learning model. These derived features were therefore generated automatically within the preprocessing pipeline using mathematical formulas. Appropriate validation, including division-by-zero checks, was also implemented to ensure safe calculations. This automated feature engineering reduced user input complexity while maintaining consistency between the training and deployment environments.

Different Imputation Strategies for Training and Deployment

Different environments required different missing value handling strategies. During model training, the complete dataset was available, allowing the use of the Iterative Imputer (MICE) to estimate missing values using relationships among multiple variables. However, during deployment, predictions were performed on individual shipment records, making iterative imputation impractical. Therefore, dataset median imputation was adopted for deployment to provide fast, deterministic, and reliable predictions. This approach improved training data quality while ensuring efficient and consistent live predictions.

Model Validation and Prediction Reliability

Before deployment, it was essential to verify that the entire prediction pipeline operated correctly and consistently. A comprehensive validation process was conducted to check the project directory structure, model artifact integrity, feature alignment, deterministic prediction behaviour, batch CSV prediction, dashboard functionality, and handling of incomplete inputs. All validation tests were successfully completed, confirming that the deployed model generated reliable and consistent predictions and that the complete prediction pipeline was ready for production use.

Conclusion and Future Work

Project Summary

The **SwiftCargo Shipment Delay Prediction System** successfully achieved its primary objective of predicting whether a shipment is likely to be delayed before the scheduled delivery date. By leveraging machine learning techniques, feature engineering, and real-time operational data, the system enables logistics organizations to identify high-risk shipments in advance and take proactive measures to minimize delays. The project demonstrates the complete lifecycle of a machine learning application, including data integration, preprocessing, exploratory data analysis, feature engineering, model development, hyperparameter tuning, feature selection, evaluation, and deployment through an interactive Streamlit web application. The developed solution provides accurate and reliable predictions while supporting data-driven decision-making in logistics and supply chain operations.

Key Achievements

The major achievements of the project are summarized below:

- Successfully integrated logistics data from 12 different data sources into a unified dataset.
- Developed a comprehensive preprocessing pipeline for handling missing values, duplicate records, outliers, categorical encoding, and feature engineering.

- Performed Exploratory Data Analysis (EDA) to identify key operational, environmental, and transportation factors influencing shipment delays.
- Evaluated multiple supervised machine learning algorithms and optimized the best-performing models using GridSearchCV.
- Applied Recursive Feature Elimination with Cross-Validation (RFECV) to identify the 30 most important features.
- Developed an optimized Random Forest Classifier that achieved:
    - Accuracy: 71.28%
    - Precision: 73.68%
    - ROC-AUC: 72.46%
- Built and deployed a production-ready Streamlit web application for real-time shipment delay prediction.
- Integrated OpenWeatherMap API to incorporate live weather information into the prediction process.

Business Value

The developed system provides significant business value by enabling organizations to:

- Shift from reactive to proactive logistics management.
- Identify high-risk shipments before dispatch.
- Improve route planning and resource allocation.
- Reduce operational and transportation costs.
- Minimize Service Level Agreement (SLA) violations.
- Improve customer satisfaction through timely interventions.
- Support data-driven operational planning and decision-making.

Limitations

Although the proposed system achieved satisfactory predictive performance, several limitations remain.

The model achieved a recall of **33.47%**, indicating that some delayed shipments are still not detected. Prediction accuracy also depends heavily on the quality and completeness of the input data. Since the application retrieves weather information using the OpenWeatherMap API, temporary API failures or inaccurate weather data may affect prediction reliability. Furthermore, the model was trained using the available SwiftCargo logistics dataset, and its performance may vary when applied to different logistics organizations, geographical regions, or operational environments.

Future Work

Several enhancements can further improve both the predictive performance and practical applicability of the proposed system.

Model Improvements

Future work may focus on improving model performance by:

- Increasing recall through class weighting and threshold optimization.
- Applying cost-sensitive learning techniques.
- Evaluating deep learning models such as Artificial Neural Networks.
- Developing ensemble models using stacking and blending approaches.
- Incorporating historical shipment trends and seasonal patterns through time-series analysis.

Feature Enhancements

Prediction accuracy can be further improved by extending the feature set through:

- Integration of real-time GPS tracking data.
- Traffic API integration.
- Carrier performance analytics.
- Additional engineered operational features.
- Natural Language Processing (NLP) for analyzing shipment notes and logistics documentation.

Application Enhancements

Future versions of the application may include:

- Automated email and SMS notifications for high-risk shipments.
- Advanced dashboards with interactive analytics and trend visualization.
- Automatic model retraining using newly collected shipment data.

- REST API development for enterprise system integration.
- Role-based authentication and secure user management.

Operational Integration

The proposed system can also be integrated with enterprise logistics platforms to support:

- Intelligent route optimization.
- Automated vehicle and driver allocation.
- Automated customer notifications regarding shipment delays.
- Continuous monitoring of prediction accuracy and operational performance through real-time dashboards.

Live Project

Project Name: SwiftCargo Shipment Delay Prediction System

Live Application:

https://swiftcargoshipmentdelayprediction-a8bcuz9uvpretde4uqo8oc.streamlit.app/

GitHub Repository:

https://github.com/Devendra-Vinod-Patil/SwiftCargo_shipment_delay_prediction

References

1. Breiman, L., "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
2. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. McKinney, W., *Python for Data Analysis*, 3rd ed., O'Reilly Media, 2022.
4. Pandas Development Team, "Pandas Documentation." Available: https://pandas.pydata.org/
5. NumPy Developers, "NumPy Documentation." Available: https://numpy.org/
6. Scikit-learn Developers, "Scikit-learn Documentation." Available: https://scikit-learn.org/
7. Streamlit Inc., "Streamlit Documentation." Available: https://docs.streamlit.io/
8. OpenWeather, "OpenWeather API Documentation." Available: https://openweathermap.org/api
9. XGBoost Developers, "XGBoost Documentation." Available: https://xgboost.readthedocs.io/
10. Python Software Foundation, "Python Documentation." Available: https://docs.python.org/3/

Appendix A: Performance Metrics

The following evaluation metrics were used to assess the performance of the machine learning models.

Accuracy: Measures the proportion of correctly classified shipments among all predictions.

Formula:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Measures the proportion of correctly predicted delayed shipments among all shipments predicted as delayed.

Formula:

$$\frac{TP}{TP + FP}$$

- **Recall:** Measures the proportion of actual delayed shipments correctly identified by the model.

Formula:

$$\frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of Precision and Recall.

Formula:

$$\frac{2 \times (Precision \times Recall)}{Precision + Recall}$$

- **ROC-AUC:** Measures the model's ability to distinguish between delayed and on-time shipments across different classification thresholds.

Confusion Matrix Terms

- **TP (True Positive):** Correctly predicted delayed shipment.
- **TN (True Negative):** Correctly predicted on-time shipment.
- **FP (False Positive):** On-time shipment incorrectly predicted as delayed.
- **FN (False Negative):** Delayed shipment incorrectly predicted as on-time.

Appendix B: Final Random Forest Hyperparameters

Hyperparameter	Value
n_estimators	100
max_depth	20
min_samples_split	2
min_samples_leaf	1
random_state	42

Appendix C: Selected Features

The final Random Forest model used the following **30 optimized features** identified through RFECV:

1. Priority
2. Weight (kg)
3. Volume (CBM)
4. Declared Value
5. Weight per Unit
6. Average Rating
7. Fleet Size
8. Years of Service
9. Vehicle Capacity (kg)
10. Maintenance Status
11. Vehicle Age
12. Vehicle Utilization
13. Warehouse Capacity
14. Current Utilization
15. Distance (km)
16. Average Transit Days
17. Route Risk
18. Traffic Index
19. Temperature
20. Rainfall
21. Humidity
22. Wind Speed
23. Visibility
24. Dispatch Lead Time
25. Expected Transit Days
26. Value Density
27. Shipment Type (Import)
28. Weather Condition (Fog)
29. Weather Condition (Storm)
30. Documentation Complete

End of Report
Thank You